

Desafios de Programação - Resolução

O que está em fundo azul, é o que se espera que o aluno faça.

1- Dada uma matriz de números inteiros A[3][3], verificar se existem elementos repetidos nessa matriz.

Segue o esqueleto do programa em Linguagem Java (o aluno pode escolher outra linguagem). O aluno deve demonstrar o funcionamento do programa alterando o valor dos elementos da matriz conforme solicitado pelo professor avaliador.

Dica: Considere a matriz como se fosse um array unidimensional e utilize um algoritmo tipo BubbleSort para comparar os elementos

```
public class RepeticaoEmMatriz {  
    public static void main(String[] args) {  
        int a[][] = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
    }  
}
```

```
// Uma solução possível  
for (int x = 0; x < 8; x++) {  
    for (int y = x + 1; y < 9; y++) {  
        if (a[x / 3][x % 3] == a[y / 3][y % 3]) {  
            System.out.println("Há elementos repetidos");  
            return;  
        }  
    }  
}  
System.out.println("Não há elementos repetidos");
```

2- Problema da conjectura de Goldbach

A conjectura de Goldbach (ainda não provada) diz que qualquer número par maior ou igual a 4 é a soma de dois números primos.

Desafio: Faça um programa que, recebendo um número P par ($4 \leq P \leq 2147483647$), seja capaz de exibir dois números inteiros correspondentes aos dois números primos cuja soma seja igual ao número par P.

Requisitos funcionais:

- Os valores de saída devem ser ordenados em ordem crescente.
 - Existindo mais de uma combinação possível, exibe-se aquela cujo primeiro valor seja o menor.
 - Não existindo valores (parabéns! você foi o primeiro no mundo que provou que a conjectura é falsa!) exiba -1.
- Lembre-se: número primo é todo número inteiro maior que 1 que somente é divisível por si próprio e pela unidade.

(adaptado de: 53 Problemas de Programação - Adonai Estrela Medrado – Professor do curso de Ciências da Computação da Universidade Federal da Bahia http://www.adonaimedrado.pro.br/documentos/53_Problemas_de_Programacao.pdf)

Dicas: Crie um método que receba um inteiro e retorne “true” se for um número primo ou “false” caso contrário. Por uma questão lógica, um dos números deve estar na primeira metade de “p” e o outro, na segunda metade.

O aluno deve demonstrar o funcionamento do programa alterando o valor de “p” conforme solicitado pelo professor avaliador.

```
// Uma solução possível

public class ConjecturaDeGoldbach {
    public static void main(String[] args) {
        // número P par (4 <= P <= 2147483647)
        int p = 300;

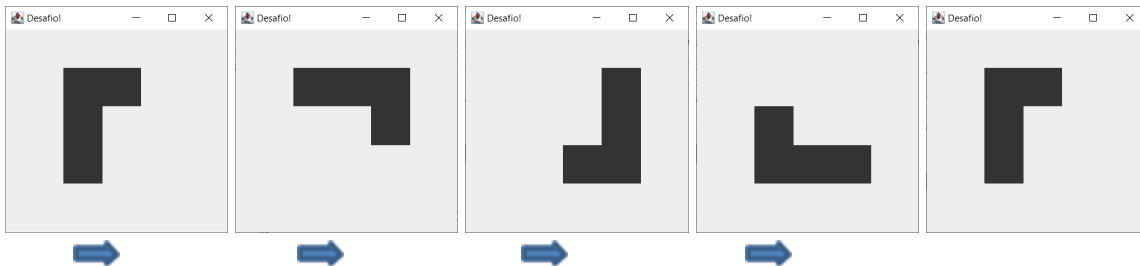
        for (int pri = 2; pri <= p / 2; pri++) {
            if (!isPrimo(pri)) continue;
            for (int seg = p / 2; seg <= p; seg++) {
                if (!isPrimo(seg)) continue;
                if (pri + seg == p) {
                    System.out.printf("Primeiro %d, Segundo %d",
                                      pri, seg);
                    return;
                }
            }
        }
        System.out.println(-1);
    }

    private static boolean isPrimo(int n) {
        if (n <= 1) return false;
        for (int cont = 2; cont <= Math.sqrt(n); cont++) {
            if (n % cont == 0) {
                return false;
            }
        }
        return true;
    }
}
```

3- Rotação de Sprite

Em alguns jogos digitais, os sprites (elementos animados) podem ser representados através de arrays bidimensionais (matrizes) e é muito comum que esses sprites possam ser rotacionados durante o jogo. Um bom exemplo é o jogo Tetris, criado pelo russo Alexey Pajitnov em 1984.

Desafio: Completar o código fornecido para que o sprite (representado por uma matriz quadrada [3x3]) seja rotacionado em 90° no sentido horário ou anti-horário, de acordo com o pressionamento das teclas “seta-direita” ou “seta-esquerda”, respectivamente.



(Marcos Mendes – Inspirado no livro A lógica do jogo: Recriando clássicos da história dos videogames - Marcus Becker – Editora Casa do Código)

Dicas: A rotação em 90° é conseguida com o auxílio de uma matriz “temporária”, copiando-se os elementos de uma **linha** da matriz original em uma **coluna** da matriz “temporária”. Primeira linha da original para a primeira coluna da temporária ou primeira linha da original para a última coluna da temporária, dependendo do sentido da rotação.

Segue o esqueleto do programa em Linguagem Java (o aluno pode escolher outra linguagem). O aluno deve demonstrar o funcionamento do programa pressionando as teclas de setas.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class RotacaoSprite extends JFrame {
    private static final long serialVersionUID = 1L;
    private final int TAMANHO = 300;
    private final Sprite sprite = new Sprite();

    public RotacaoSprite() {
        add(sprite, BorderLayout.CENTER);
        setTitle("Desafio!");
        setSize(TAMANHO, TAMANHO);
        setLocationRelativeTo(null);
        setVisible(true);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        addKeyListener(new Ouvinte());
    }

    public static void main(String[] args) {
        new RotacaoSprite();
    }
}
```



```
private class Ouvinte extends KeyAdapter {
    @Override
    public void keyPressed(KeyEvent e) {
        super.keyPressed(e);
        switch (e.getKeyCode()) {
            case KeyEvent.VK_LEFT:
                sprite.giraAntiHorario();
                break;
            case KeyEvent.VK_RIGHT:
                sprite.giraHorario();
                break;
        }
    }
}

private class Sprite extends JPanel {
    private static final long serialVersionUID = 1L;
    final int ORDEM = 3;
    int[][] desenho = {
        { 1, 1, 1 },
        { 1, 0, 0 },
        { 0, 0, 0 }
    };

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        final int MARGEM_ESQ = 75;
        final int MARGEM_SUP = 50;
        final int TAMANHO_BLOCO = 50;
        for (int i = 0; i < ORDEM; i++) {
            for (int j = 0; j < ORDEM; j++) {
                if (desenho[i][j] == 1) {
                    g.fillRect(MARGEM_ESQ + i * TAMANHO_BLOCO,
                               MARGEM_SUP + j * TAMANHO_BLOCO,
                               TAMANHO_BLOCO, TAMANHO_BLOCO);
                }
            }
        }
    }

    void giraAntiHorario() {
        final int[][] temp = new int[ORDEM][ORDEM];
        // Código para rotacionar a matriz no sentido
        anti-horário.
        for (int i = 0; i < ORDEM; i++) {
            for (int j = 0; j < ORDEM; j++) {
                temp[j][ORDEM - i - 1] = desenho[i][j];
            }
        }
        desenho = temp;
    }
}
```



```
        repaint();
    }

    void giraHorario() {
        final int[][] temp = new int[ORDEM][ORDEM];
        // Código para rotacionar a matriz no sentido horário.
        for (int i = 0; i < ORDEM; i++) {
            for (int j = 0; j < ORDEM; j++) {
                temp[ORDEM - j - 1][i] = desenho[i][j];
            }
        }
        desenho = temp;
        repaint();
    }
}
```